1. Título

Creación de Base de Datos Relacional y Política de Seguridad para el Sistema Web Progresivo LUPSI

2. Introducción

Este documento formaliza la implementación del modelo de datos relacional del sistema LUPSI sobre PostgreSQL, desplegado mediante la plataforma Supabase. Se detallan la arquitectura del esquema, las extensiones habilitadas, la estrategia de seguridad mediante Row Level Security (RLS) y los mecanismos de integridad que previenen colisiones de agendamiento médico.

La base de datos constituye el núcleo del sistema: toda lectura y escritura pasa por sus políticas de seguridad antes de ser entregada al cliente, garantizando que ninguna capa de la aplicación pueda eludir los controles de privacidad definidos.

3. Tecnología Utilizada

Componente	Descripción
PostgreSQL 15	Motor de base de datos relacional objeto, alojado en Supabase Cloud.
Supabase	Plataforma BaaS (Backend as a Service) que provee PostgreSQL gestionado, Auth, y Data API automática.
uuid-ossp v1.1	Extensión para generación de identificadores únicos universales (UUID v4) como claves primarias.
btree_gist v1.7	Extensión que habilita índices GIST sobre tipos de datos escalares, necesaria para la restricción de exclusión de rangos temporales.
Row Level Security (RLS)	Mecanismo nativo de PostgreSQL que aplica reglas de acceso a nivel de fila, garantizando que cada rol vea únicamente los datos que le corresponden.

Tabla 1: Stack tecnológico de la base de datos LUPSI.

4. Modelo Relacional

El esquema de datos está compuesto por cinco tablas en el esquema `public` de PostgreSQL, vinculadas mediante claves foráneas en cascada. La tabla `profiles` actúa como raíz de identidad, vinculada directamente con el sistema de autenticación de Supabase (`auth.users`).

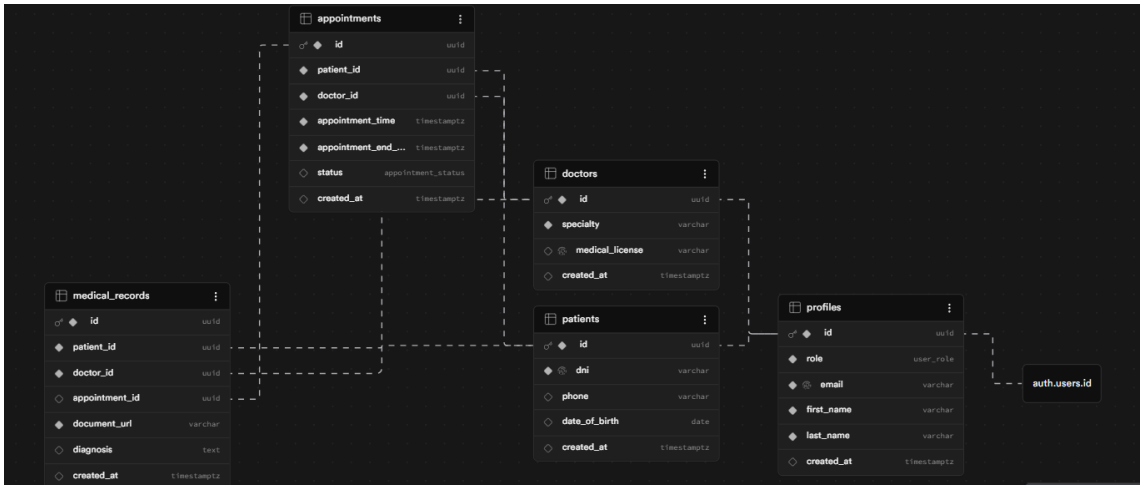


Figura 1: Diagrama relacional del esquema `public` — Supabase Schema Visualizer.

5. Descripción de Tablas y Campos

5.1. Tabla: `profiles` (Identidad y Control de Acceso)

Centraliza los datos de identidad de todos los actores del sistema. Su campo `role` determina los privilegios de acceso en toda la aplicación.

Campo	Tipo	Descripción
`id`	UUID (PK)	Referencia directa a `auth.users.id`. CASCADE en eliminación.
`role`	ENUM `user_role`	Rol del usuario: `PATIENT`, `DOCTOR`, `RECEPTIONIST`, `ADMIN`.
`email`	VARCHAR(255)	Correo electrónico único. Requerido.
`first_name`	VARCHAR(100)	Nombre del usuario.
`last_name`	VARCHAR(100)	Apellido del usuario.
`created_at`	TIMESTAMPTZ	Fecha de registro automática.

Tabla 2: Campos de `profiles`.

5.2. Tabla: `patients` (Datos del Paciente)

Extiende `profiles` con información médica específica del paciente ecuatoriano.

Campo	Tipo	Descripción
`id`	UUID (PK/FK)	Referencia a `profiles.id`.
`dni`	VARCHAR(10)	Cédula ecuatoriana. Único, validado con expresión regular `^[0-9]{10}$`.
`phone`	VARCHAR(20)	Teléfono de contacto. Opcional.
`date_of_birth`	DATE	Fecha de nacimiento. Opcional.
`created_at`	TIMESTAMPTZ	Fecha de registro automática.

Tabla 3: Campos de `patients`.

5.3. Tabla: `doctors` (Perfil Médico)

Complementa a `profiles` con las credenciales profesionales del médico.

Campo	Tipo	Descripción
`id`	UUID (PK/FK)	Referencia a `profiles.id`.
`specialty`	VARCHAR(100)	Especialidad médica. Requerido.
`medical_license`	VARCHAR(50)	Número de licencia médica. Único.
`created_at`	TIMESTAMPTZ	Fecha de registro automática.

Tabla 4: Campos de `doctors`.

5.4. Tabla: `appointments` (Citas Médicas)

Núcleo del motor de agendamiento. Incluye la restricción de exclusión temporal anti double-booking.

Campo	Tipo	Descripción
`id`	UUID (PK)	Generado automáticamente.
`patient_id`	UUID (FK)	Referencia a `patients.id`.
`doctor_id`	UUID (FK)	Referencia a `doctors.id`.
`appointment_time`	TIMESTAMPTZ	Inicio de la cita.
`appointment_end_time`	TIMESTAMPTZ	Fin de la cita. Calculado por el backend.
`status`	ENUM `appt_status`	`SCHEDULED`, `CANCELLED`, `COMPLETED`.
`created_at`	TIMESTAMPTZ	Fecha de creación.

Tabla 5: Campos de `appointments`.

5.5. Tabla: `medical_records` (Historial Clínico)

Almacena los diagnósticos y tratamientos emitidos por un doctor. Acceso bloqueado para `RECEPTIONIST` a nivel de base de datos.

Campo	Tipo	Descripción
`id`	UUID (PK)	Generado automáticamente.
`patient_id`	UUID (FK)	Referencia a `patients.id`.
`doctor_id`	UUID (FK)	Referencia a `doctors.id`.
`appointment_id`	UUID (FK)	Referencia a `appointments.id`. Opcional.
`document_url`	VARCHAR(500)	URL del recurso digital asociado.
`diagnosis`	TEXT	Diagnóstico médico en texto libre.
`created_at`	TIMESTAMPTZ	Fecha de registro.

Tabla 6: Campos de `medical_records`.

6. Extensiones PostgreSQL Habilitadas

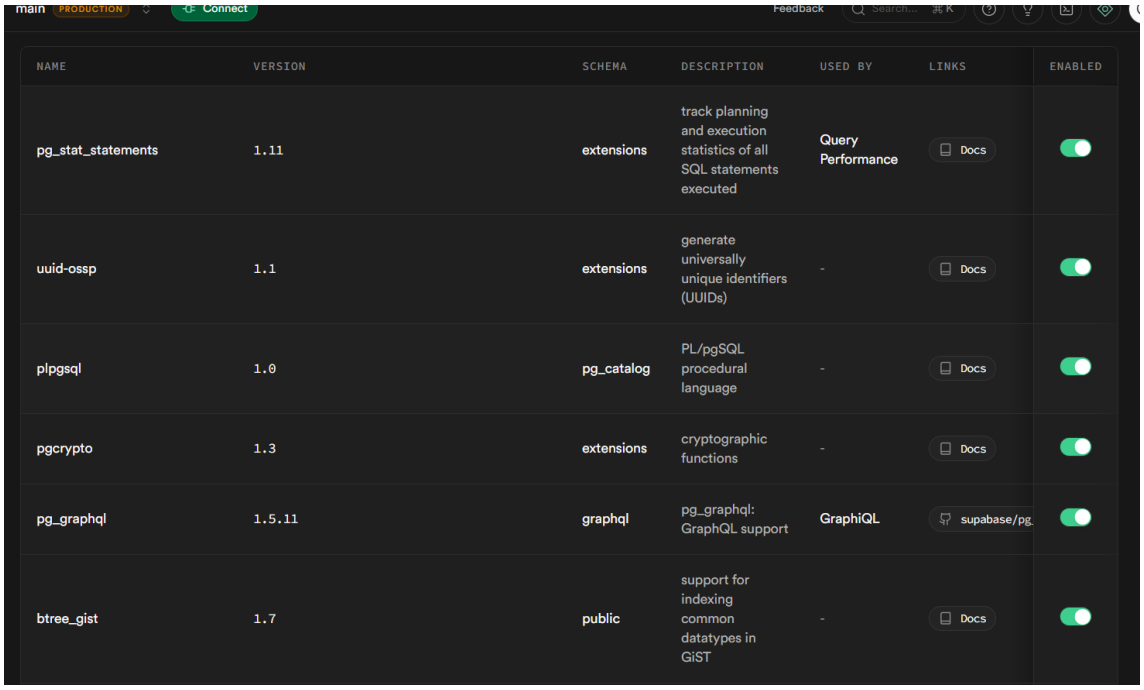


Figura 2: Panel de extensiones de Supabase — `uuid-ossp` y `btree_gist` habilitadas.

Las dos extensiones clave instaladas para LUPSI son:

- `uuid-ossp` **v1.1** — Provee la función `uuid_generate_v4()` usada como valor por defecto en las claves primarias de `appointments` y `medical_records`. Garantiza unicidad criptográfica distribuida sin dependencia de secuencias numéricas.
- `btree_gist` **v1.7** — Extiende el soporte de índices GiST para tipos de datos estándar (UUID, TIMESTAMPTZ). Es un requisito indispensable para que la cláusula `EXCLUDE USING gist` de la tabla `appointments` pueda comparar el campo `doctor_id` (UUID) junto al rango temporal.

7. Índices y Restricciones de Integridad

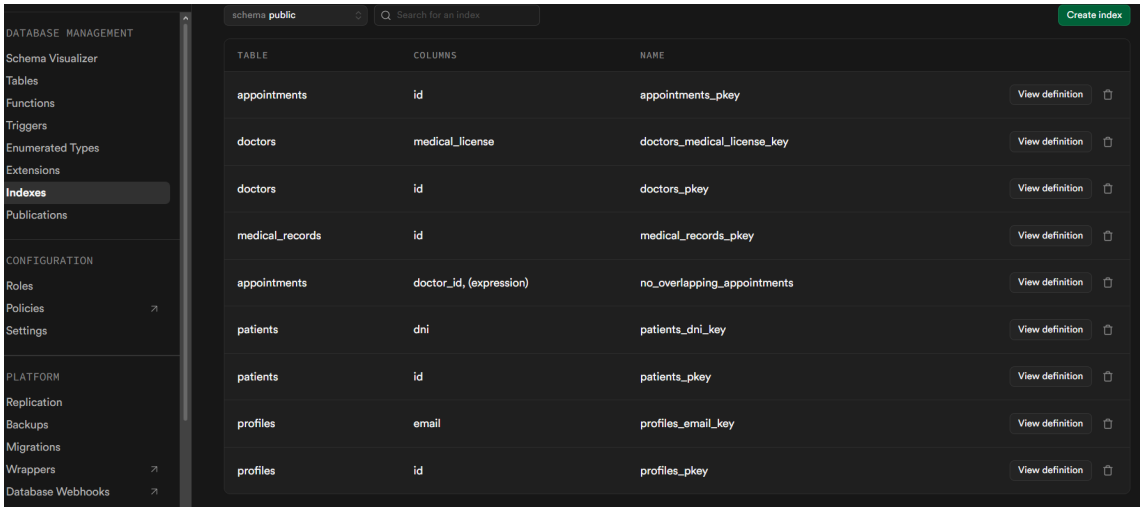


Figura 3: Panel de índices del esquema `public` — índice `no_overlapping_appointments` visible.

7.1. Índices Automáticos (Claves Primarias y Únicas)

PostgreSQL genera automáticamente un índice B-tree por cada restricción `PRIMARY KEY` y `UNIQUE`. Los índices registrados son:

Tabla	Nombre del Índice	Propósito
`appointments`	`appointments_pkey`	Clave primaria UUID.
`doctors`	`doctors_pkey`	Clave primaria UUID.
`doctors`	`doctors_medical_license_key`	Unicidad del número de licencia médica.
`medical_records`	`medical_records_pkey`	Clave primaria UUID.
`patients`	`patients_pkey`	Clave primaria UUID.
`patients`	`patients_dni_key`	Unicidad del número de cédula (10 dígitos).
`profiles`	`profiles_pkey`	Clave primaria UUID.
`profiles`	`profiles_email_key`	Unicidad del correo electrónico.

Tabla 7: Índices B-tree generados automáticamente por restricciones de unicidad.

7.2. Índice GiST de Exclusión Temporal — Anti Double-Booking

El índice más crítico del sistema es `no_overlapping_appointments`. Es un índice GiST compuesto definido mediante la cláusula `EXCLUDE USING gist`:

- **Columnas:** `doctor_id` (con operador `=`) + `tstzrange(appointment_time, appointment_end_time)` (con operador `&&`).
- **Condición:** Solo aplica cuando `status = 'SCHEDULED'`, ignorando las citas canceladas o completadas.
- **Efecto:** Si dos transacciones simultáneas intentan insertar citas con el mismo doctor en un rango horario solapado, PostgreSQL lanza una excepción de exclusión. El backend NestJS captura esta excepción y retorna `HTTP 409 Conflict`.

8. Modelo de Seguridad: Row Level Security (RLS)

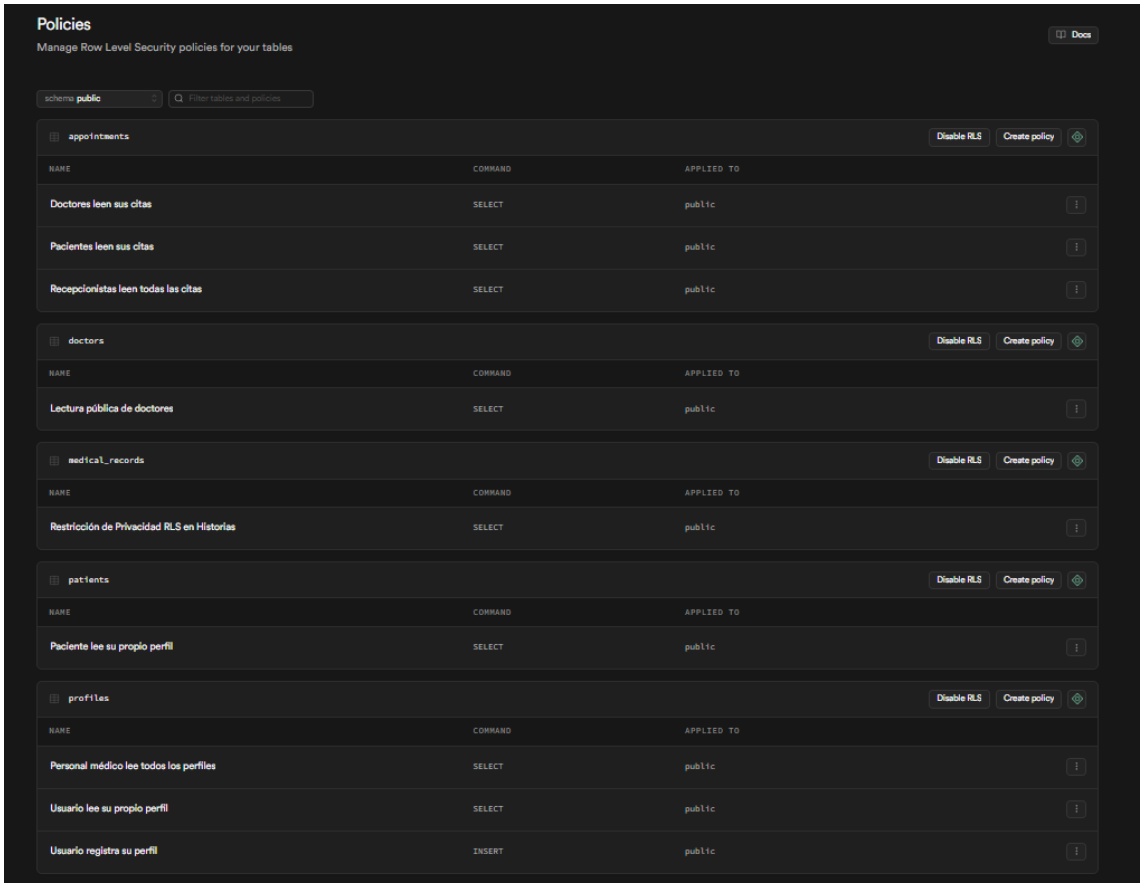


Figura 4: Panel de políticas RLS del esquema `public` — 7 políticas activas en 5 tablas.

RLS está habilitado en las cinco tablas del sistema (`ALTER TABLE ... ENABLE ROW LEVEL SECURITY`). Esto significa que sin una política de `SELECT`, `INSERT`, `UPDATE` o `DELETE` que lo autorice explícitamente, ningún rol puede acceder a ninguna fila, independientemente de los permisos a nivel de columna o tabla.

8.1. Arquitectura de Doble Barrera

LUPSI implementa seguridad en dos capas independientes y acumulativas:

1. **Capa de Aplicación (NestJS Guards):** El backend verifica el JWT y el rol del usuario antes de procesar cualquier solicitud. Un rol no autorizado recibe `HTTP 403 Forbidden` antes de llegar a la base de datos.
2. **Capa de Datos (PostgreSQL RLS):** Aunque una solicitud llegue a la capa de datos, las políticas RLS garantizan que cada consulta solo devuelva las filas correspondientes al usuario autenticado.

8.2. Inventario de Políticas RLS

Tabla	Nombre de Política	Cmd	Condición
`appointments`	Doctores leen sus citas	SELECT	`auth.uid() = doctor_id`
`appointments`	Pacientes leen sus citas	SELECT	`auth.uid() = patient_id`
`appointments`	Recepcionistas leen todas las citas	SELECT	rol = `RECEPTIONIST`
`doctors`	Lectura pública de doctores	SELECT	usuario autenticado
`medical_records`	Restricción de Privacidad RLS en Historias	SELECT	solo paciente o doctor propio
`patients`	Paciente lee su propio perfil	SELECT	`auth.uid() = id`
`profiles`	Usuario lee su propio perfil	SELECT	`auth.uid() = id`
`profiles`	Personal médico lee todos los perfiles	SELECT	rol = `DOCTOR/RECEPTIONIST`
`profiles`	Usuario registra su perfil	INSERT	`auth.uid() = id`

Tabla 8: Inventario completo de políticas RLS del sistema LUPSI.

8.3. Caso Crítico: Aislamiento de Historias Clínicas

La política *"Restricción de Privacidad RLS en Historias"* en la tabla `medical_records` garantiza que el rol `RECEPTIONIST` jamás pueda leer datos clínicos. Este bloqueo opera en la capa de datos: aunque un recepcionista realice una solicitud directa a la

API de Supabase con su JWT válido, la consulta devolverá cero filas porque ninguna política lo autoriza.

9. Tipos de Datos Enumerados (ENUMs)

Se definieron dos tipos de datos enumerados para garantizar la consistencia de los valores de estado en el sistema:

- `public.user_role`: `PATIENT` | `DOCTOR` | `RECEPTIONIST` | `ADMIN` — controla los privilegios de acceso.
- `public.appointment_status`: `SCHEDULED` | `CANCELLED` | `COMPLETED` — define el ciclo de vida de una cita médica.

Los ENUMs previenen la inserción de valores inválidos y simplifican la lógica de filtrado en las políticas RLS.

10. Conclusiones

La base de datos del sistema LUPSI implementa una estrategia de seguridad de defensa en profundidad (defense-in-depth) con las siguientes garantías:

1. **Privacidad garantizada a nivel de fila:** RLS activo en las 5 tablas con 9 políticas que cubren todos los roles del sistema.
2. **Integridad de agendamiento:** El índice GiST `no_overlapping_appointments` previene el doble agendamiento mediante una restricción de exclusión temporal atómica.
3. **Identidad centralizada:** La tabla `profiles` vincula el sistema de autenticación de Supabase con la lógica de roles, evitando fragmentación de identidad.
4. **Validación en origen:** La restricción `CHECK (dni ~ '^[0-9]{10}$')` garantiza que ningún DNI inválido pueda persistirse sin pasar por la validación del Módulo 10.

11. Firmas de Responsabilidad

Rol	Nombre / Representante	Firma (QR)	Fecha
Gestor del Proyecto	Angel Ayuquina		27/03/2026
Arquitecto BD y Cloud	Sebastián Ortiz		27/03/2026
Desarrollador Backend y QA	Daniel Luisa		27/03/2026
Desarrollador Frontend y UX/UI	Alex Guachi		27/03/2026